Package management on Linux

>>Learn the easy way to package your software into .deb and .rpm like a boss > by Ankit Mathur

Traditionally, Linux being an open source platform, users have been restricted to developers and software enthusiasts who prefer to compile packages from source. But as more and more normal users continue to adopt this platform as an alternative to Windows, package managers have emerged as a new breed which have become all the more important to differentiate how each platform manages and keeps track of where individual files and packages are installed on a Linux system.

Package management in Linux is usually present as an all in one solution to managing and updating everything related to your applications, and unlike Windows, Linux package repositories contain almost every software package that you might ever need. This not only provides a one stop solution for updating all the installed software, but also tracks and manages dependencies that each package might have. Dependency resolution is one beast that can be very difficult to tackle if done manually. Modern package managers however, make this a breeze. Today, Linux software repositories resemble more like an app store for mobile platforms, where you just give the command to install something and it gets done without you having to worry about any installation wizard. All thanks to the package manager under Linux, which knows exactly which files and configurations belong to a particular package.

It is not very difficult to get up and running with creating packages on Linux. Among others, there are two widely popular package management systems that we will be discussing i.e. Debian based and Red Hat/Fedora based.

Debian based packaging

Debian packages are used in Debian based Linux distros, where a binary software package is available in the form of a .deb file. This file contains within itself all the metadata related to the package.

The dpkg utility is the main package management program which performs all the primary functions of installing removing and providing information about the .deb files. Mainly one can interact with deb packages using this command as follows: -

- Installing a deb package:
  `dpkg -i debfilename`
- Removing an installed package:
  `dpkg -r packagename`

- List of installed packages:
  `dpkg -l`
- Query whether a package is installed or not:
  `dpkg -l 'packagename'`
- Determine the package name to which a given file belongs to:
  `dpkg -S /path/to/file`
- Show metadata about a package:
  `dpkg -s packagename`

Apt or Advanced Packaging Tool is designed to work on top of the dpkg tool and further simplifies the task of package management by performing higher level functions like sourcing of deb files from their repositories and managing the relations between multiple packages. Apt is more like a collection of tools where the apt-get utility being the most important one, works with package names, unlike dpkg which works with deb file names. When a package is needed to be installed, it automatically resolves all the required dependencies if they are available in its list of online package repositories. One can use the following commands to interact with apt: -

- Install and update software on the system:
  `apt-get install packagename`
- Remove software:
  `apt-get remove packagename`
- Update package list:
  `apt-get update`
- Upgrade the system:
  `apt-get upgrade`
- Search for a package:
  `apt-cache search packagename`